To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp("ls", "ls", "-l", NULL);

        printf("Exec failed!\n");
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 1$ gcc task1.c -o task1
geya@LenovoL480:~/OS/Skill/Week 1$ ./task1
Parent Process
Parent PID: 807
Child Process
Child PID: 808
Child PID: 808
Parent PID: 807
total 36
-rwxr-xr-x 1 geya geya 16208 Aug 14 08:59 task
-rwxr-xr-x 1 geya geya 16208 Aug 14 09:00 task1
-rw-r--r-- 1 geya geya   701 Aug 14 08:58 task1.c
Child process completed.
```

```
geya@LenovoL480:~/OS/Skill/Week 1$ strace ./task1
execve("./task1", ["./task1"], 0x7ffdd7555ac0 /* 26 vars */) = 0
brk(NULL)                               = 0x5f41e0c40000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7cc5c4de4000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20085, ...}) = 0
mmap(NULL, 20085, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7cc5c4ddf000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \250\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7cc5c4a00000
mmap(0x7cc5c4a28000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7cc5c4a28000
mmap(0x7cc5c4bc0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c0000) = 0x7cc5c4bc0000
mmap(0x7cc5c4c0e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20d000) = 0x7cc5c4c0e000
mmap(0x7cc5c4c14000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7cc5c4c14000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7cc5c4ddc000
arch_prctl(ARCH_SET_FS, 0x7cc5c4ddc740) = 0
set_tid_address(0x7cc5c4ddcd68)         = 534
set_robust_list(0x7cc5c4ddca20, 24)     = 0
rseq(0x7cc5c4ddc6a0, 0x20, 0, 0x53053053) = 0
mprotect(0x7cc5c4c0e000, 16384, PROT_READ) = 0
mprotect(0x5f41a18e0000, 4096, PROT_READ) = 0
```

```
geya@LenovoL480:~/OS/Skill/Week 1$ pstree
systemd─┬─agetty
        ├─chronyd-starter───chronyd───chronyd
        ├─cron
        ├─dbus-daemon
        ├─init-systemd(Ub─┬─SessionLeader───Relay(299)───bash───pstree
        │                 ├─init───{init}
        │                 ├─login───bash
        │                 └─{init-systemd(Ub}
        ├─networkd-dispat
        ├─rsyslogd───3*[{rsyslogd}]
        ├─systemd───(sd-pam)
        ├─systemd-journal
        ├─systemd-logind
        ├─systemd-resolve
        ├─systemd-timedat
        ├─systemd-udevd
        ├─unattended-upgr───{unattended-upgr}
        └─wsl-pro-service───7*[{wsl-pro-service}]
```